

Relazione Progetto Lab. Applicazioni Mobili

Ultiwear

Realizzata da Alessandro Nanni

Email: alessandro.nanni17@studio.unibo.it

Matricola: 0001027757

Indice

1 Overview dell'app	1
1.1 Note tecniche	1
1.2 Primo avvio dell'app	2
2 Guardaroba/Wardrobe	3
3 Esplora/Browse	4
4 Eventi/Upcoming Events	5
5 Scambi/Trades	5
5.1 Matches	6
5.2 Manual Trade	8
5.3 Quick Trade	8
5.4 Storico Scambi	10
6 Notifiche	10
6.1 Quando è stato aggiunto un nuovo torneo	10
6.2 Quando un post raggiunge i 5 like	11
6.3 Giorni mancanti ai prossimi 3 tornei a cui si partecipa	11

1 Overview dell'app

L'applicazione Ultiwear è progettata per giocatori di frisbee con la passione per lo scambio di capi d'abbigliamento.

Tramite essa è possibile visualizzare i propri capi in un armadio digitale, pubblicarli, mostrare interesse per eventuali scambi, consultare i tornei futuri e parteciparvi, e gestire scambi in tempo reale.

Dato che l'app non vuole sostituire il momento dello scambio in persona, e lasciare un pò di mistero e magia, non ci sono username pubblici e non si ha modo di comunicare con gli altri.

1.1 Note tecniche

Quasi tutti i dati utilizzati sono salvati in un database *firebase*: questo include sia le rappresentazioni in forma di tabelle di oggetti, che le immagini.

L'app è realizzata con *jetpack compose*: non ci sono file XML contenenti *view* nelle cartelle *res*.

L'autenticazione è effettuata tramite account Google.

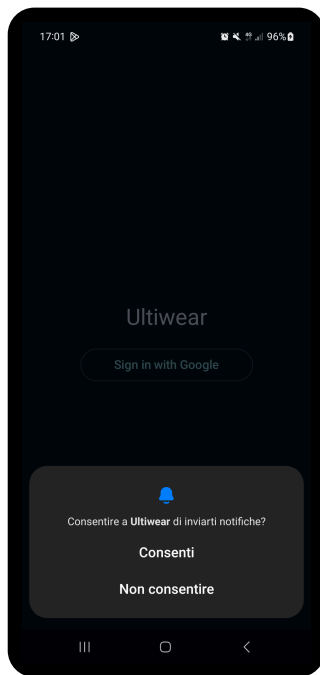
Come consigliato dalla [documentazione ufficiale](#), l'app utilizza una singola attività, la navigazione tra le diverse sezioni è gestita tramite lo stato di Compose.

Si cerca di rispettare il modello MVVM, sono presenti *package* con compiti precisi:

- data: comunicazione con il database;
- model: contiene le data class
- notifications: si occupa dell'invio di notifiche;
- view: contiene i *composable*;
- viewModel: contiene i *ViewModel* intermediari tra View e Model

L'app utilizza il *MaterialTheme* di Android.

1.2 Primo avvio dell'app



La prima volta che si avvia l'app verranno richiesti i permessi necessari: invio di notifiche, posizione e accesso alla fotocamera, richiesti dal manifest.

AndroidManifest.xml

```
1 <uses-permission
  android:name="android.permission.CAMERA" />
2 <uses-permission
  android:name="android.permission.POST_NOTIFICATIONS"/
  >
3 <uses-permission
  android:name="android.permission.ACCESS_FINE_LOCATION
  >
```

Nella *MainActivity*, questi vengono richiesti in catena affinché l'ultimo (*locationPermission*), non sovrascriva i precedenti.

```
1 private fun requestPermissionsOnStartup() {
2     notificationPermissionLauncher.launch(POST_NOTIFICATIONS)
3 }
4 private val notificationPermissionLauncher =
5     registerForActivityResult(ActivityResultContracts.RequestPermission()) { _ ->
6         requestCameraPermission() // next
7 }
```

Listing 1: Dopo aver richiesto il permesso per le notifiche, si chiederà quello per la fotocamera. Successivamente, tramite stati *composable* dell' *authViewModel*, si controlla che l'utente sia registrato e connesso a internet.



```

1 GetCredentialRequest.Builder().addCredentialOption(
2     GetGoogleIdOption.Builder()
3     .setFilterByAuthorizedAccounts(false)
4     .setServerClientId(BuildConfig.GOOGLE_CLIENT_ID)
5     .setAutoSelectEnabled(false)
6     .build()).build()

```

Listing 2: Codice per creare il prompt “sign in with Google” e ottenere un token di autenticazione

In base a questi stati si mostreranno 3 *view* diverse. Dopo aver eseguito l’accesso con Google, l’utente potrà utilizzare l’app.

Il token di login sarà poi convertito in un token Firebase, memorizzato in una collezione come UID assieme alla mail dell’utente. Ora l’utente ha accesso all’app, e potrà navigare le sue 5 *view*.

`var selectedIndex by rememberSaveable { mutableIntStateOf(0) }` tiene traccia della *sub-activity* corrente. Ognuna di esse è definita dal *composable* `TabItem`.

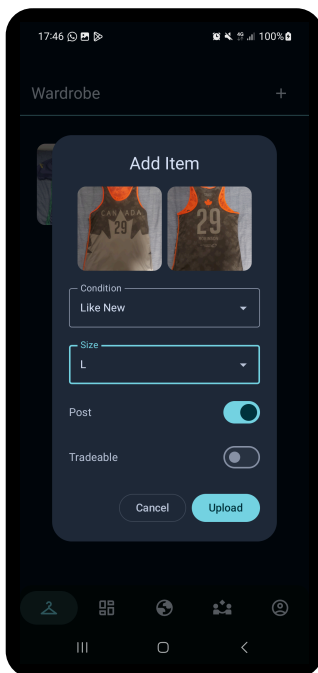
```

1 val tabs = listOf(
2     TabItem(
3         "Wardrobe", R.drawable.wardrobe
4     ) {
5         WardrobeScreen(wardrobeViewModel)
6     },
7     ...
8 )

```

2 Guardaroba/Wardrobe

Qui l’utente può inserire dati e foto relativi ai suoi capi d’abbigliamento.



Per ogni capo si può fare una foto del fronte, retro, specificare taglia e condizioni, e infine scegliere se pubblicare¹ e indicare se questo capo lo si vuole scambiare. I capi sono mostrati tramite una `LazyVerticalGrid` di 3 colonne. Cliccando un elemento della griglia si potranno visualizzare i dettagli del capo.

Le operazioni sul database sono effettuate tramite l’oggetto `ItemHandler`, di cui il `WardrobeViewModel` possiede un riferimento. Un `listener` controlla continuamente cambiamenti nella collezione `wardrobe` per gli oggetti dell’owner. In caso di cambiamenti si aggiornano 3 liste di item: una che li contiene tutti, una che contiene solo quelli scambiabili e un’ultima che contiene solo quelli posseduti (`items.filter { it.owned }`). Il `listener` permette di aggiornare automaticamente gli *item* dell’utente in ogni punto dell’app (*Wardrobe*, *Browse* e *Trade*).

Ogni oggetto capo contiene la variabile booleana `owned`. Questa è usata per indicare se il capo deve essere mostrato nell’armadio, nel caso l’utente corrente non ne sia più il proprietario in seguito ad uno scambio².

¹Visibile nella sezione *Browse*, mostrata in seguito.

²Eseguibile nella sezione *Quick Trade*, mostrata in seguito.

Inizialmente per gestire la cancellazione di un capo si eliminava il documento corrispondente. Dopo aver aggiunto i post e timeline degli scambi, è stato utilizzato un metodo che non elimina i propri capi, ma li nasconde solamente. Il post associato invece viene eliminato, ma dato che l'item originale esiste ancora sul database, vi si può fare riferimento per mostrare gli item dati via in uno scambio.

Le foto degli item (e come si vedrà in seguito, anche quelle degli scambi) vengono modificate prima di essere caricate sul database. Una singola foto scattata occupa dai 3.5MB ai 4MB. Dato che Firebase fornisce 1GB di memoria prima di far pagare per ogni megabyte occupato, per sicurezza ho implementato due metodi che riducono la dimensione dell'immagine.

```
1 val baos = ByteArrayOutputStream()
2 bitmap.compress(Bitmap.CompressFormat.WEBP, 80, baos)
3 val bytes = baos.toByteArray()
```

Questo metodo, dopo aver ricavato la bitmap dell'immagine scattata con la fotocamera, converte l'immagine in formato WEBP³ con una qualità pari all'80% di quella originale. Infine i byte dell'immagine vengono caricati su Firebase.

3 Esplora/Browse



In questa sezione si possono vedere le foto e dettagli degli *item* postati. Per ogni post non proprio si può anche mettere like e se il proprietario lo ha permesso, esprimere interesse di scambio. I bottoni di scambio e like sono oscurati e non cliccabili per i propri capi d'abbigliamento. Ogni post è contenuto in una *LazyColumn*, ed è una *Card* composta da un *Pager* che si può scorrere per vedere le foto di fronte e retro e una *InfoBar*.

Il *BrowseViewModel* contiene un riferimento allo stesso handler, e dunque listener del di *Wardrobe*. Dunque, quando un utente preme il bottone like, il *ViewModel* viene notificato di questo cambiamento e ricarica la colonna. Per evitare che un utente possa mettere like allo stesso post più volte, ogni post dispone di una collezione che contiene gli id degli utenti che hanno messo like.

Similmente, la collezione *trade_interests* tiene traccia degli utenti che hanno espresso interesse a scambiare un certo item.

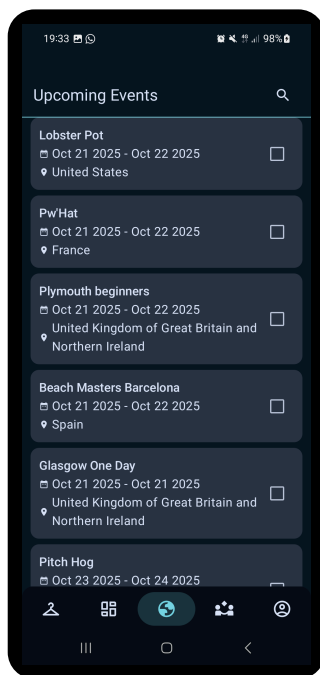
```
1 try {
2     val newLikes = handler.toggleLike(wardrobeUid, currentUser.uid)
3     // update state
4     _items.value = _items.value.map { item ->
5         if (item.wardrobeUid == wardrobeUid) {
6             item.copy(post = item.post?.copy(likes = newLikes))
7         } else item
8     }
9 } catch (e: Exception) {
10     Log.e(tag, "Error toggling like", e)
```

³Le immagini WEBP hanno dimensione minore di quelle in formato PNG o JPEG.

I like possono essere messi e tolti: il handler si occupa di aggiornare il database con una transazione per rendere atomica l'operazione di lettura e scrittura. Il `ViewModel` crea una copia degli *items*, e se l'id di uno di essi corrisponde con quello a cui è stato cambiato il like, si aggiorna il valore mostrato nella *card*.

4 Eventi/Upcoming Events

Qui si possono vedere i prossimi tornei da tutto il mondo, ottenuti tramite l'api di [Ultical](#). Sono *card* ordinate dalla data di inizio più vicina alla più lontana. Ogni card ha associata una *checkbox* per indicare se si sarà presenti ad un certo torneo.



Il singleton `APIClient` inizializza una sola volta (tramite `by lazy`) l'interfaccia `UlticalAPI` fornendogli l'url a cui fare la chiamata HTTP, e una factory da usare per processare i dati. In questo caso si usa la libreria GSON di Google in quanto l'API di `ultical` restituisce un oggetto JSON. Retrofit è la libreria usata per fare chiamate HTTP.

```
1 val api: UlticalApi by lazy {
2     Retrofit.Builder()
3         .baseUrl(URL)
4         .addConverterFactory(
5             GsonConverterFactory.create()
6         )
7         .build()
8         .create(UlticalApi::class.java)
9 }
```

L'interfaccia fornisce un metodo `getEvents()` per ottenere una lista di eventi.

Quando viene aperta questa sezione per la prima volta, prima si esegue un “flattening” dell'oggetto JSON: dato che un torneo ha una lista di edizioni, ciascuna con un ID, si esegue questa operazione per lavorare su dati più omogenei. Nella lista appiattita vengono inclusi solamente gli eventi con data di inizio maggiore o uguale a quella attuale.

Quando un utente si dichiara presente ad un torneo, non solo si aggiorna la collezione su Firebase, ma un *listener* sulla collezione `user_attendances` modifica la `MutableMap` `attendances`, che per ogni ID, associa un booleano per indicare se l'utente sarà presente ad un certo torneo. Questo esempio evidenzia perfettamente l'utilità del `ViewModel`: qualora ci fosse bisogno di ricomporre lo schermo, non è necessario fare una nuova chiamata all'API, dal momento che i valori sono già memorizzati `EventViewModel`.

Quando si apre nuovamente la schermata eventi dopo aver chiuso l'app, le presenze sono ripristinate sulla *view* perché ogni *card* dispone di un riferimento al `ViewModel`, che può usare per controllare se l'utente è presente al torneo con id pari a quello della *card* che si sta componendo.

5 Scambi/Trades

In questa *view* si possono vedere e fare tutte le operazioni relative agli scambi, interagendo direttamente o indirettamente con altri utenti.

La sezione degli scambi è costituita da 3 *view*. Quella attualmente in vista è controllata da una state variable di *compose*

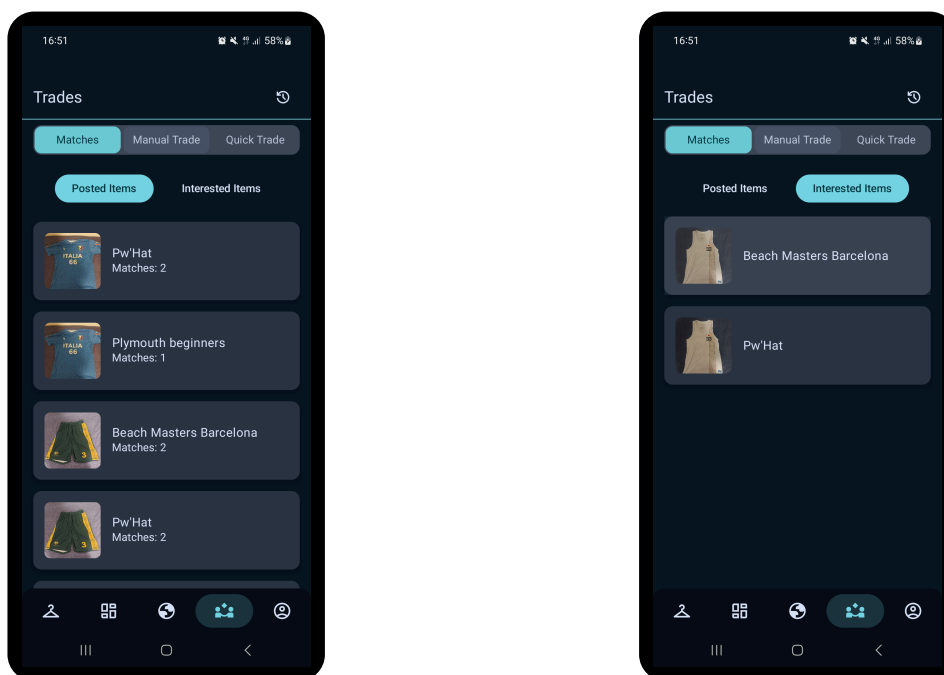
```
1 var selectedTab by remember { mutableIntStateOf(0) }
2 val tabs = listOf("Matches", "Manual Trade", "Quick Trade")
```

kt

Per ogni tab, se la sua posizione nella lista coincide con l'indice, il suo sfondo viene colorato per evidenziare che è quella selezionata. Un altro stato, `showTradesDialog` (booleano) indica se bisogna mostrare il dialogo che contiene lo storico degli scambi.

5.1 Matches

In questa sezione è possibile visualizzare i tornei a cui parteciperanno gli utenti che hanno espresso interesse per un determinato capo, o per i quali l'utente stesso ha mostrato interesse.



In *Posted Items* puoi vedere quante persone sono interessate agli oggetti che hai pubblicato e che parteciperanno a un torneo a cui sarai presente anche tu. Ad esempio, due persone che vogliono scambiare la maglia dell'Italia saranno a *Pw'Hat*, a cui ci sarò anch'io. In *Interested Items* puoi invece visualizzare gli oggetti per cui hai espresso interesse, i cui proprietari saranno presenti a un torneo in comune con te. Ad esempio, l'utente proprietario della canottiera del giappone per la quale ho espresso interesse a scambiare sarà, come me, ai *Beach Masters Barcelona*.

Anche in questa *view*, uno stato `var showIncoming by remember { mutableStateOf(false) }` si occupa di indicare quale sezione sarà visibile.

La struttura è gestita da un unico componente *composable*, il quale, in base al valore della variabile `showIncoming`, determina quale delle due liste utilizzare per popolare il contenuto. L'assegnazione avviene tramite `displayMatches = if (showIncoming) incomingMatches else matches`.

TradeMatchesViewModel si occupa di fare gli incroci tra i tornei e gli oggetti che gli utenti vogliono scambiare. Dato che richiede le gli eventi e gli *item* di un utente, esso dispone di riferimenti ai *ViewModel* delle sezioni *browse* ed *events*. Prima di calcolare gli incroci, si attende che i valori dei *ViewModel* richiesti siano pronti.

```
1 viewModelScope.launch {
2     combine(
```

kt

```

3      snapshotFlow { browseViewModel.items.value.isNotEmpty() },
4      snapshotFlow { eventViewModel.events.isNotEmpty() }
5  ) { browseReady, eventsReady ->
6      browseReady && eventsReady
7  }
8      .filter { it }
9      .first()
10     loadPostedMatches()
11     loadInterestedMatches()
12 }

```

Viene avviata una coroutine legata al ciclo di vita di TradeMatchesViewModel. All'interno di essa, gli stati osservabili dei ViewModel dipendenti (browseViewModel ed eventViewModel) vengono convertiti in Flow, affinché si possano osservare le variazioni dei loro valori. Le due Flow vengono combinate tramite l'operatore combine, in modo da produrre un unico flusso che emette un valore booleano ogni volta che uno dei due stati cambia. Il valore emesso è true solo quando entrambi i ViewModel hanno completato il caricamento dei propri dati. Infine si filtrano solo i valori true, e il metodo first() sospende la coroutine fino alla prima emissione valida. Quando entrambi i ViewModel hanno terminato il caricamento, la coroutine riprende l'esecuzione ed esegue le funzioni loadPostedMatches() e loadInterestedMatches(), il calcolo delle combinazioni è delegato a TradeHandler.

Per quanto riguarda ricavare gli utenti interessati ai propri capi si eseguono i seguenti passi:

1. da Firebase, seleziona tutti i trade_interests associati ad oggetti che l'utente possiede;
2. mappa l'id di un item a una liste di id di utenti interessati, usata per fare controlli più veloci (tradeMap);
3. ottieni una lista di utenti interessati a oggetti che l'utente corrente possiede, rimuovendo duplicati;
4. mappa le coppie <userID, tournamentID> a un booleano per indicare se un utente parteciperà a un certo torneo;
5. per ogni oggetto pubblicato, recupera gli utenti interessati dalla tradeMap;
6. per ogni torneo a cui l'utente partecipa, conta quanti interessati partecipano a quel torneo;
7. se c'è ne è almeno 1, aggiungi quel torneo e il numero di interessati ai valori da restituire;

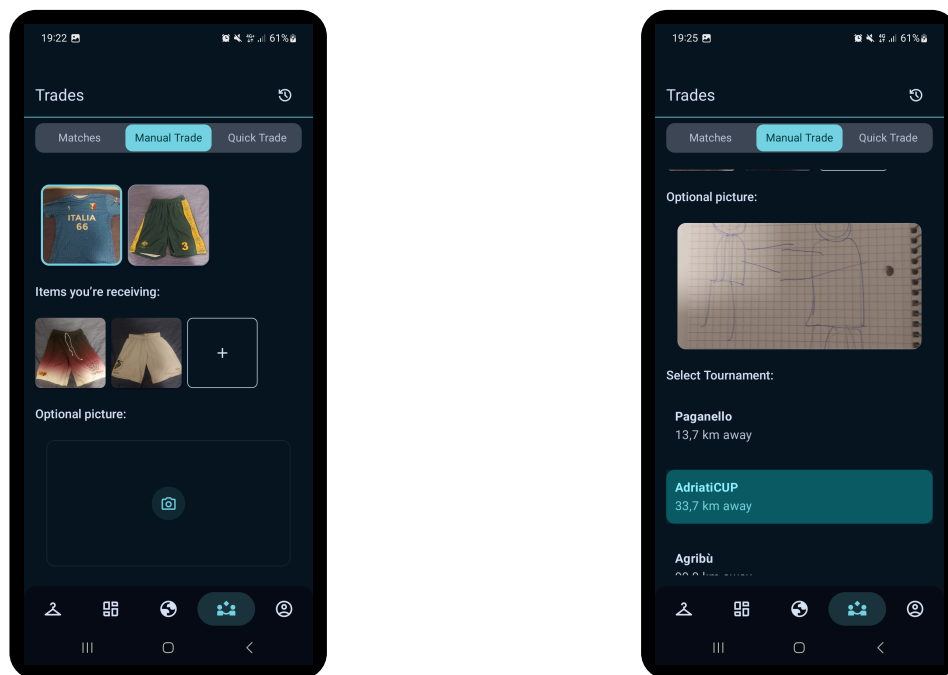
Per ricavare i tornei in cui saranno presenti i proprietari dei capi per cui si ha espresso interesse di scambio si eseguono passaggi simili:

1. si ottengono i trade_interests dell'utente da Firebase;
2. si estraggono gli id degli oggetti interessati;
3. controllo aggiuntivo per ogni item per assicurarsi che esista ancora;
4. selezione tornei a cui partecipa l'utente in una mappa <tournamentID, boolean>;
5. preparazione di una mappa <<ownerId, tournamentId>, Boolean> (attendanceMap) che indica se un proprietario sarà presente o meno ad un torneo specifico;
6. per ogni proprietario:
 1. si recuperano i tornei a cui esso partecipa;
 2. si aggiorna attendanceMap per ogni torneo di quell'utente;
7. per ogni oggetto interessato:
 1. controlla ogni torneo a cui l'utente partecipa;
 2. verifica se il proprietario dell'item sarà presente;
 3. in caso positivo, recupera i dettagli del torneo e aggiungili a quelli da restituire.

5.2 Manual Trade

La sezione *manual trade* permette di rimuovere rapidamente uno o più *item* dal proprio guardaroba, e aggiungerne altri in un'ipotetica situazione di scambio. Si potranno selezionare gli item da dare via, e quelli ricevuti. Inoltre è possibile scattare una foto e selezionare l'evento presso cui è stato fatto lo scambio tra una lista dei 3 più vicini per località.

Questo tipo di scambio è a quello rapido, che verrà mostrato in seguito, se l'utente con cui si sta scambiando non ha l'app.



La maglia con il bordo azzurro è l'item che ho deciso di dare via, i due pantaloni sono gli item che sto ricevendo. Scorrendo in basso è possibile scattare una foto del momento, e selezionare il torneo dove è stato fatto lo scambio. Per ottenere i 3 eventi più vicini è abbastanza facile, dato che l'api fornisce latitudine e longitudine di quasi tutti i tornei.

1. dalla lista degli eventi si eliminano quelli senza latitudine e longitudine;
2. per ogni torneo:
 1. prepara calcola la distanza tra la posizione attuale e il torneo;
 2. crea una coppia `<tournament, distance>`.
3. ordina le coppie in base alla distanza;
4. prendi le prime 3.

Dato che la distanza viene calcolata in metri, nella card viene formattata per mostrare i chilometri.

5.3 Quick Trade

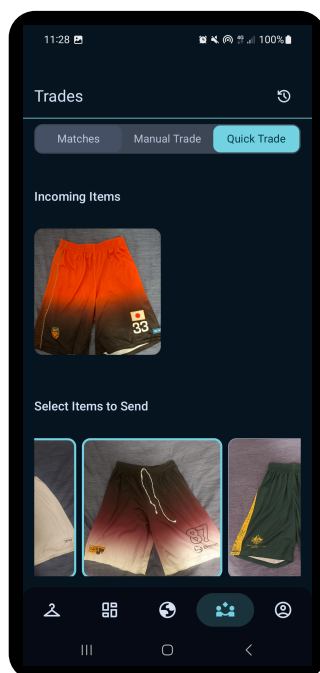
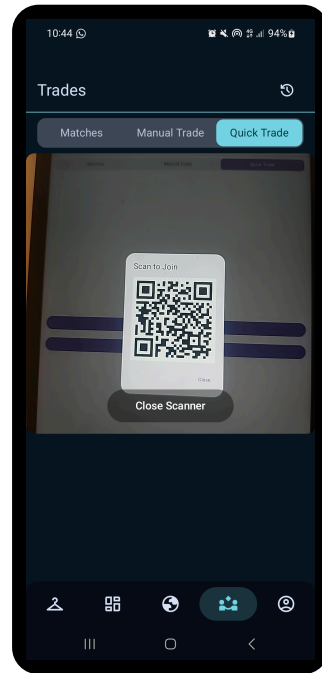
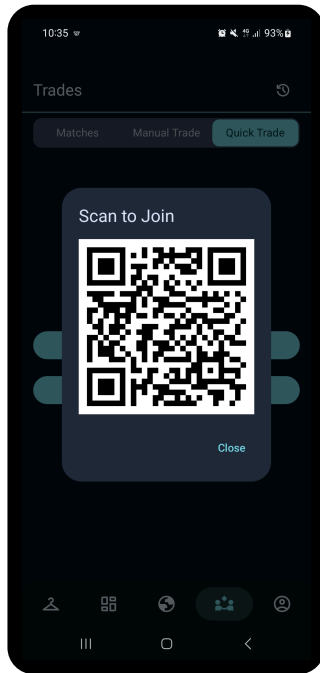
Lo scambio rapido può essere svolto tra utenti che possiedono l'app. Si crea una sessione di scambio che per entrambi permette di selezionare gli *item* che si stanno dando via e quelli che si stanno ricevendo.

Un utente inizierà la sessione di scambio, generando un *qr code*. L'altro potrà usare uno scanner per inquadrare il *qr code* e dare via alla sessione di scambio. Prima di creare il *qr code*, il `ViewModel` inserisce nel database una *trade_session*, caratterizzata da creatore, partecipanti, data di creazioni e un booleano che indica se è pronta, ovvero se il *qr code* è stato scannerizzato. Inizializzata a `false`. Verrà restituito anche un id corrispondente alla sessione iniziata, che verrà convertito in un *qr code*. Per fare ciò si usa un metodo che converte la stringa in una matrice di bit, adottando il formato del *qr code*. In seguito si itera per righe e per colonne su ogni bit di questa matrice, e dove si trova un bit

true si disegna un pixel nero, altrimenti bianco.

Per la scansione dei *qrcode* viene utilizzato un blocco `LaunchedEffect`, che inizializza e configura la fotocamera tramite *CameraX*⁴. Quest'ultima consente di visualizzare in tempo reale il flusso video all'interno di un componente `PreviewView`. Attraverso il componente `ImageAnalysis`, ogni fotogramma catturato viene analizzato in tempo reale mediante la libreria *ML Kit*, che si occupa del riconoscimento dei *qrcode*.

Quando l'analisi ha successo, *ML Kit* restituisce una lista di potenziali codici. Ognuno di essi viene controllato e, se valido, viene chiamata la funzione `onQrScanned(it)`, dove `it` rappresenta la stringa decodificata dal *qrcode*. Questa funzione funge da callback e invoca un metodo del `ViewModel` che consente all'utente che ha effettuato la scansione di entrare nella relativa sessione di scambio.



Il `ViewModel` modifica il documento corrispondente per inserire l'utente nei membri della sessione e dare via alla sessione di scambio, dove si potrà vedere in tempo reale gli *item* che entrambi le parti vogliono scambiare.

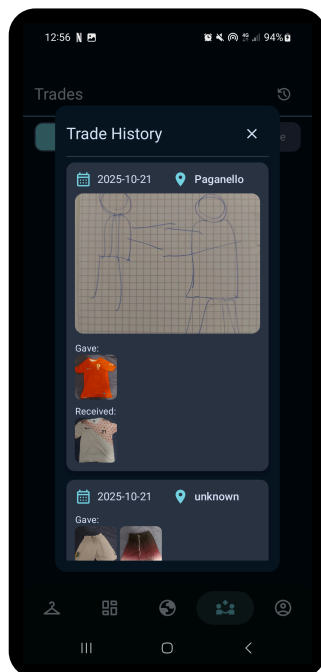
Si possono selezionare dal proprio guardaroba gli item da dare, e quelli non dichiarati come scambiabili non saranno visibili. Si possono inoltre vedere gli item che si stanno ricevendo.

Quando entrambi confermano, gli item verranno trasferiti automaticamente nei corrispettivi nuovi armadi. Per finalizzare lo scambio, si imposta `confirmedBySender` o `confirmedByReceiver` nel database a true, quando entrambi sono veri si creano copie degli item scambiati, dove l'id dei proprietari vengono invertiti. In questo modo si ha un riferimento all'oggetto scambiato associato a se, anche l'utente a cui è stato dato lo scambia nuovamente o elimina.

I dati relativi allo scambio vengono caricati una sola volta sul database, sarà poi il `TradeHandler` a determinare quali oggetti mostrare in base all'id dell'utente.

⁴Liberia per Android Jetpack che semplifica l'integrazione della fotocamera nelle app Android.

5.4 Storico Scambi



Qui si possono vedere gli scambi fatti in passato, ordinati in ordine cronologico a partire dal più recente. Per ogni scambio si può vedere la data, torneo, la foto scattata e gli oggetti ceduti e ricevuti.

Il `TradeHandler` ottiene una lista di scambi, dove l'utente corrente è o `userA` o `userB`. Per ognuno di questi, si crea un oggetto `Trade`. Questo insieme di liste viene usato dal `TradeViewModel` per avere accesso nella view `Trades`, indipendentemente dallo stato della pagina, agli scambi passati.

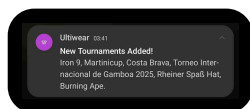
6 Notifiche

L'app può inviare 3 tipi di notifiche periodicamente. Nella `MainActivity` viene creato il canale usato per la ricezione delle notifiche.

6.1 Quando è stato aggiunto un nuovo torneo

Sempre nella `MainActivity` viene definito il worker che controlla ogni ora se ci sono nuovi tornei.

```
1 val tournamentCheckRequest =  
2   PeriodicWorkRequestBuilder<TournamentCheckWorker>(  
3     1, TimeUnit.HOURS  
4   ).build()  
5  
6   WorkManager.getInstance(this).enqueueUniquePeriodicWork(  
7     "tournament_check",  
8     ExistingPeriodicWorkPolicy.KEEP,  
9     tournamentCheckRequest  
10  )
```

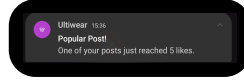


Dato che i `ViewModel` sono collegati al lifecycle dei *composable*, non possono essere usati per queste operazioni in background. Nel metodo `doWork()` del `CoroutineWorker` associato a questo lavoro, si eseguono i seguenti passi:

1. carica i dati salvati nel work precedente nella *preferences* (*storage* XML che contiene coppie chiave-valore);
2. tra questi, seleziona gli id dei tornei salvati;
3. esegui la chiamata all'API;
4. per ogni evento ed edizione:
 1. se inizia in un giorno dopo oggi;
 2. aggiungi il suo id a quelli da salvare

3. salva in una lista i dati di quel torneo (salvati in un oggetto `TournamentUI`, già usato nella view `Events`)
5. se questa lista non è vuota, invia una notifica con i nomi di questi tornei;
6. salva la *preference* con gli id dei nuovi tornei,

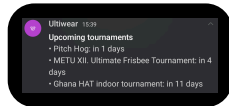
6.2 Quando un post raggiunge i 5 like



Come per la notifica precedente, il controllo periodico è fatto da un `Worker`. Nel suo metodo `doWork()`:

1. carica tutti i post dell'utente e li converte in oggetti;
2. se il post ha un numero di like $\geq$ a 5, e la notifica associata a questo post non è stata inviata
 1. invia la notifica;
 2. imposta il campo `notification` a `true` per indicare che la notifica è già stata mandata

6.3 Giorni mancanti ai prossimi 3 tornei a cui si partecipa



Ogni volta che un utente dichiara la sua presenza ad un torneo dalla sezione `Events`, viene chiamato il metodo `metodo`

`NotificationScheduler.scheduleDailyReminder(context)`, che dopo un delay per assicurarsi che inizi a mezzogiorno, avvia un `Worker` che ogni giorno informa l'utente di quanti giorni mancano ai suoi 3 tornei più vicini.

Nel metodo `doWork()`, si invoca l'API per ottenere la lista dei tornei, i dati vengono convertiti in una lista di tornei, ignorando quelli passati, e si selezionano quelli a cui l'utente ha dichiarato di partecipare. Tra questi, si calcola la differenza tra il giorno attuale e il giorno in cui essi iniziano, e in base a questo valore si ordina la lista. Infine si selezionano i nomi dei primi 3 tornei, con associati i giorni che mancano. Questi sono i valori che verranno comunicati dalla notifica.